Beginner Guide to Git Tags and Semantic Versioning

Tagging lets you bookmark exact moments in your code history so deployments stay reliable and reproducible. Learning semantic versioning ensures your teammates and users always understand what changed in each release.

What you'll learn:

1. Understanding Semantic Versioning (MAJOR.MINOR.PATCH)
2. Creating Annotated Release Tags
3. Publishing Tags to Remote Repositories
4. Inspecting and Testing Older Releases

Aman Raj Singh
Cloud Operations Engineer @ iManage
linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Understanding Semantic Versioning (MAJOR.MINOR.PATCH)

Semantic Versioning follows a simple three-number pattern: MAJOR.MINOR.PATCH. You increment PATCH when you release backward-compatible bug fixes. You increment MINOR when you add new features that don't break existing functionality. You increment MAJOR only when you make breaking changes that alter how users interact with your code.

```
$# Example: Moving from patch fix to new feature
 git tag -a v1.0.1 -m "Fix typo in billing form"
 git tag -a v1.1.0 -m "Add dark mode toggle"
```

2

TIP 2 OF 4

Creating Annotated Release Tags

Git supports two types of tags: lightweight and annotated. Always use annotated tags for real project releases. Annotated tags store the creator's name, email, date, and a custom release note directly in Git's database. This provides an immutable audit trail for compliance and deployment pipelines.

```
$# Create an annotated tag with a description message
 git tag -a v1.0.0 -m "Production launch release v1.0.0"
 git show v1.0.0
```

3

TIP 3 OF 4

Publishing Tags to Remote Repositories

Running a standard git push command transfers commits and branches, but it ignores tags by default. To trigger your GitHub Actions or GitLab CI/CD release workflow, you must explicitly push your tag to the remote server. You can push tags individually or push all local tags simultaneously.

```
$# Push a single tag (safest practice)
 git push origin v1.0.0

 # Push all local tags at once
 git push origin --tags
```

4

TIP 4 OF 4

Inspecting and Testing Older Releases

When a customer reports an issue on an older version, you need to test that exact snapshot. You can check out any tag just like a branch to enter a detached HEAD state. If you need to make changes, simply branch off from that specific tag.

```
$# Inspect files at tag v1.0.0
 git checkout v1.0.0

 # Create a fix branch directly from the tag
 git checkout -b hotfix-v1.0.0-auth v1.0.0
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Always prefix version tags with a lowercase 'v' (e.g., v1.0.0) for consistency.
+ Use annotated tags (`-a`) for production releases instead of plain lightweight tags.
+ Include a concise release summary in your tag message explaining key changes.
+ Push tags individually to avoid accidentally publishing unfinished local draft tags.
+ Connect Git tags to CI/CD pipelines to automate container builds and deployments.

Watch Out For

! Assuming git push uploads your tags automatically without explicitly specifying them.
! Modifying or moving an existing tag after other teammates have already pulled it.
! Bumping a MAJOR version for small bug fixes instead of a simple PATCH increment.
! Working directly on a detached HEAD state after checking out a tag without creating a branch.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh